Computer Networks Assignment 1 Report

Krishnan R (CS24BTECH11036), Praneeth Kodavati (CS24BTECH11034)

Contents

1 Introduction **2**

2 Question 1 – Host-name Resolution **2**

2.1 Objective . . . 2
2.2 Core Idea . . . 2
2.3 Implementation . . . 2
2.4 Screenshots . . . 3

3 Question 2 – Networked Messaging System **4**

3.1 Overview . . . 4
3.2 Architecture . . . 4
3.3 Implementation . . . 4
3.3.1 Server and Client . . . 4
3.3.2 IPv4 Multicast . . . 5
3.3.3 IPv6 and IPv6 Multicast . . . 5
3.3.4 Group Management . . . 6
3.3.5 Message Timestamps . . . 6
3.3.6 SSL/TLS . . . 6
3.3.7 Dynamic Group Ports . . . 7
3.3.8 IPv4 and IPv6 Coexistence . . . 7
3.4 Testing the system . . . 7
3.4.1 Group Creation . . . 7
3.4.2 Join and Leave . . . 8
3.4.3 IPv4 Communication . . . 8
3.4.4 IPv6 Communication . . . 8
3.4.5 Mixed IPv4 and IPv6 Communication . . . 8
3.4.6 Testing the SSL/TLS part . . . 9

4 Conclusion **9**

5 References **9**

1 Introduction

In this assignment, we built a networked messaging system using socket-based communication. The system has a server and multiple clients, and it covers hostname resolution, TCP communication, UDP multicast, IPv4 and IPv6, SSL/TLS, timestamps, and group management.

We separate the control part from the data part. TCP handles reliable communication between a client and the server for operations such as creating, joining, and leaving groups. Chat messages are sent using UDP multicast to the members of a group.

Both IPv4 and IPv6 clients are supported, along with SSL/TLS (encrypted TCP communication). An IPv4 and an IPv6 client can belong to the same logical group, but their multicast traffic is sent through separate IPv4 and IPv6 multicast channels.

2 Question 1 – Host-name Resolution

2.1 Objective

The first part of the assignment is about resolving a hostname to a suitable network address using the standard socket API.

2.2 Core Idea

We use `getaddrinfo()` to resolve the hostname and service name. Rather than handling IPv4 and IPv6 resolution separately, the application asks the operating system for a list of suitable socket addresses.

By setting the address family to `AF_UNSPEC`, the application can accept both IPv4 and IPv6 addresses.

2.3 Implementation

```
struct addrinfo hints{};
struct addrinfo *result = nullptr;

hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;

int status = getaddrinfo(hostname, port, &hints, &result);

if (status != 0) {
    // Print error message
}
```

The function returns a linked list containing the possible addresses. The client can go through these addresses and try to establish a connection.

Using `getaddrinfo()` makes the implementation independent of whether a hostname resolves to an IPv4 or IPv6 address. It also avoids hard-coding assumptions about the underlying address family.

2.4 Screenshots

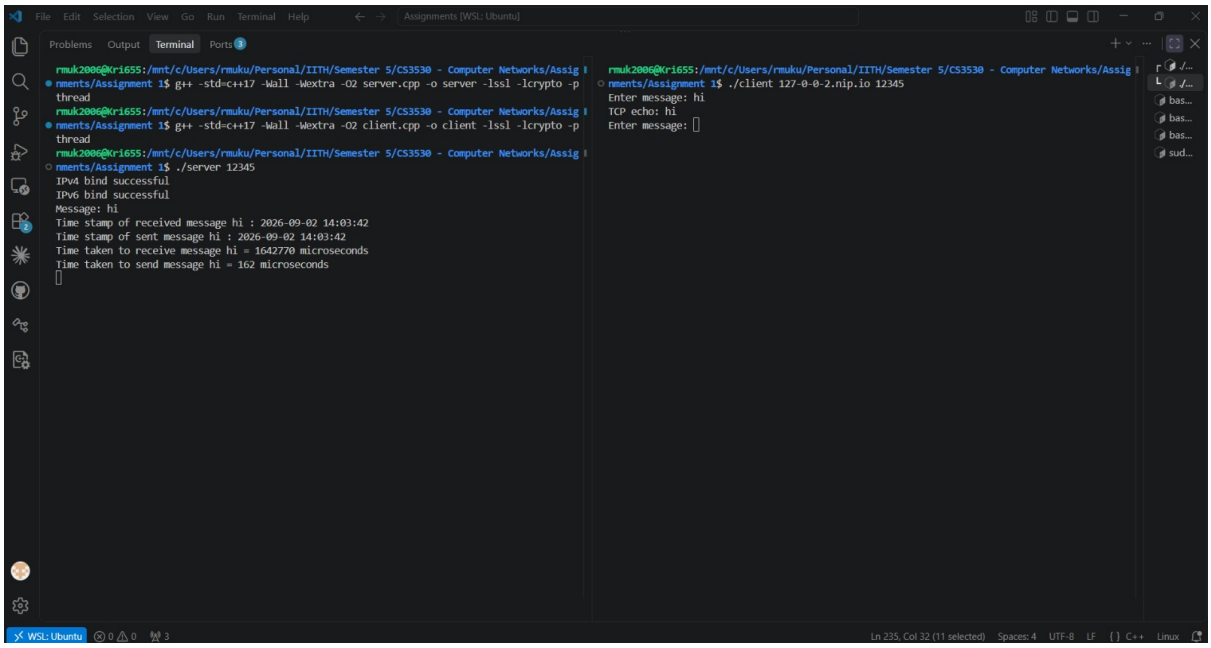


Figure 1: Screenshot of successful execution of the client software with hostname

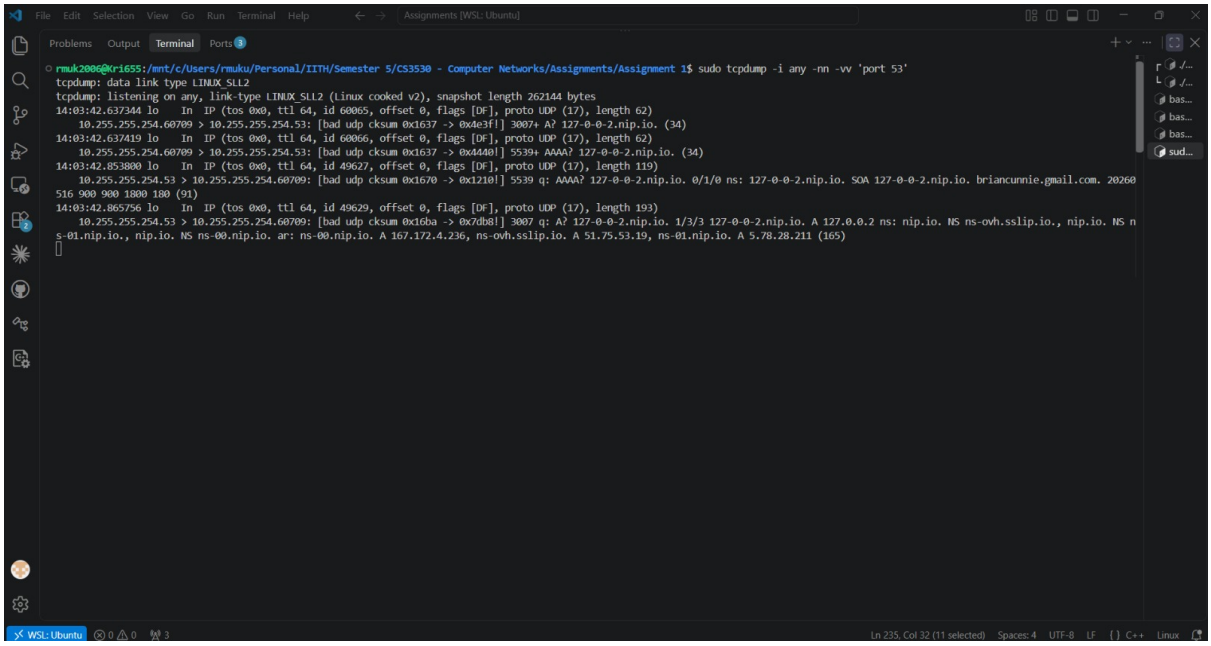


Figure 2: Screenshot of Wireshark/tcpdump showing DNS query and response

3 Question 2 – Networked Messaging System

3.1 Overview

The main system consists of one server and multiple clients. We use TCP as the control part and UDP multicast as the data part.

The messaging system also includes the following features:

1. Message timestamps.
2. SSL/TLS-secured TCP communication.
3. IPv4 multicast messaging.
4. IPv6 communication and IPv6 multicast.
5. Group creation and membership management.

3.2 Architecture

TCP is used for group-management operations. Clients contact the server when they want to create a group or join or leave an existing one.

After joining a group, the client uses UDP multicast to send and receive messages. The sender therefore does not need a separate connection to every member of the group.

The overall architecture is:

Client
↓ TCP
Server
↓ group information
Multicast Group
↑ UDP multicast
Clients

The server keeps track of the logical groups and their IPv4 and IPv6 multicast ports. After creating or joining a group, a client receives the multicast information it needs from the server.

3.3 Implementation

3.3.1 Server and Client

The server listens for TCP connections from clients and handles the group-management commands it receives.

The server is started using:

```
./server 12344
```

For an IPv4 client, we use:

```
./client localhost 12346
```

For an IPv6 client, we use:

```
./client localhost 12347
```

The same commands are available to both IPv4 and IPv6 clients:

- `create`
- `join N`
- `leave N`

3.3.2 IPv4 Multicast

For IPv4 multicast, we use the following multicast address:

```
239.0.0.1
```

The client creates a UDP socket and joins the multicast group using the corresponding multicast socket option.

With IPv4 multicast, multiple clients can receive messages sent to the same multicast address and port. The sender does not have to send a separate packet to each client.

3.3.3 IPv6 and IPv6 Multicast

For IPv6, we use the `AF_INET6` address family, with IPv6 socket addresses represented by the `sockaddr_in6` structure.

The IPv6 multicast address used by the implementation is:

```
ff02::1234
```

For link-local IPv6 multicast, we also need to specify the network interface. We obtain its interface index using `if_nametoindex()`.

Example:

```
unsigned int ifindex = if_nametoindex("eth0");
```

This interface index is then associated with the IPv6 socket.

The client joins an IPv6 multicast group using `IPV6_JOIN_GROUP`. When it leaves the group, the membership is removed using `IPV6_LEAVE_GROUP`.

Conceptually:

```
setsockopt(sock,
           IPPROTO_IPV6,
           IPV6_JOIN_GROUP,
           ...);
```

and:

```
setsockopt(sock,
           IPPROTO_IPV6,
           IPV6_LEAVE_GROUP,
           ...);
```

3.3.4 Group Management

The `create` command creates a new logical group. The server assigns the multicast ports for that group and sends the group information back to the client.

The server returns a response in the following form:

```
CREATED groupNo ipv4Port ipv6Port
```

The `join N` command asks the server for the multicast information associated with group N. The client then uses this information to join the corresponding multicast group. The server then returns the multicast information:

```
JOINED groupNo ipv4Port ipv6Port
```

The client uses this information to join the appropriate IPv4 or IPv6 multicast group.

The `leave N` command informs the server that the client is leaving the logical group. The client then removes its corresponding multicast membership.

3.3.5 Message Timestamps

The TCP echo service records timestamps for received and sent messages. The server records the time at which a message is received and the time at which the response is sent, along with the time taken for these operations.

3.3.6 SSL/TLS

We added SSL/TLS support to the TCP communication service on port `12345`. The OpenSSL library is used to establish an encrypted TCP connection between the client and server.

The connection works in the following sequence:

1. Establish a TCP connection to port `12345`.
2. Initialize the SSL/TLS context.

3. Perform the TLS handshake.
4. Exchange messages over the encrypted connection.
5. Echo the received messages back to the client.

TLS can be enabled by passing the `-ssl` option to the client and server.

We also implemented SSL/TLS-secured TCP communication on the TCP service.

3.3.7 Dynamic Group Ports

Each logical group gets its own multicast ports.

We use the following scheme:

$$P_{IPv4} = IPV4_GROUP_BASE_PORT + 2 \times groupNo, \quad (1)$$
$$P_{IPv6} = IPV6_GROUP_BASE_PORT + 2 \times groupNo. \quad (2)$$

This gives each logical group a different set of multicast ports.

3.3.8 IPv4 and IPv6 Coexistence

IPv4 and IPv6 clients can belong to the same logical group.

Each logical group has two multicast endpoints:

- one IPv4 multicast endpoint;
- one IPv6 multicast endpoint.

An IPv4 client uses the IPv4 endpoint for sending and receiving, while an IPv6 client uses the IPv6 endpoint.

Thus, IPv4 and IPv6 clients can be in the same logical group even though their multicast traffic is kept separate.

3.4 Testing the system

We tested the system in multiple ways like:

3.4.1 Group Creation

We started the server using:

```
./server 12344
```

We then connected a client and issued:

```
create
```

The server returned the group number along with the IPv4 and IPv6 multicast ports.

3.4.2 Join and Leave

A second client was connected to the server and given the command:

```
join 1
```

The client joined group 1 successfully.

The client was then able to leave the group using:

```
leave 1
```

After leaving, the client no longer received multicast messages from that group.

3.4.3 IPv4 Communication

We placed two IPv4 clients in the same logical group.

A message sent by one IPv4 client was received successfully by the other client.

This showed that IPv4 multicast was working correctly.

3.4.4 IPv6 Communication

We also placed two IPv6 clients in the same logical group.

A message sent by one IPv6 client was received successfully by the other client.

This showed that IPv6 communication and multicast group membership were working correctly.

3.4.5 Mixed IPv4 and IPv6 Communication

We then placed an IPv4 client and an IPv6 client in the same logical group.

The observed behavior was:

Table 1: Mixed IPv4 and IPv6 communication results.

Communication	**Result**
IPv4 → IPv4	Successful
IPv6 → IPv6	Successful
IPv4 → IPv6	Not received
IPv6 → IPv4	Not received

The last two results are expected since IPv4 and IPv6 multicast use different protocols and multicast endpoints.

Even though they cannot receive each other's multicast packets, both clients are still members of the same logical group maintained by the server.

3.4.6 Testing the SSL/TLS part

The client and server completed the TLS handshake successfully. Messages were then exchanged successfully over the encrypted TCP connection on port `12345`.

4 Conclusion

We implemented a working networked messaging system with support for both IPv4 and IPv6 communication.

TCP is used for reliable control-plane communication, while UDP multicast handles the actual message distribution. The server manages group creation, joining, and leaving, and multicast is used for messages within a group.

IPv6 support was added using the IPv6 socket API and IPv6 multicast. We also implemented SSL/TLS-secured TCP communication on the TCP service. The systemwas tested with multiple clients, including a combination of IPv4 and IPv6 clients.

Through this assignment, we worked with socket programming, multicast communication, IPv6 networking, TLS, and a concurrent server design in a practical setting.

5 References

1. Linux man-pages project, `getaddrinfo(3)`.
2. Linux man-pages project, Linux socket interface documentation.
3. Linux man-pages project, Linux IPv4 protocol documentation.
4. Linux man-pages project, Linux IPv6 protocol documentation.
5. Linux man-pages project, Linux UDP documentation.
6. Linux man-pages project, `if_nametoindex(3)`.
7. OpenSSL Documentation, OpenSSL documentation.
8. OpenSSL Documentation, `SSL_new()` documentation.
9. Beej's Guide to Network Programming, Beej's Guide to Network Programming.